

Benefits of TensorFlow.js

Listed below are some of the benefits of TensorFlow.js:

- It supports WebGL, out-of-the-box.
- It supports GPU acceleration and therefore, behind the scenes, will accelerate your code when a GPU is available.
- All data stays on the client, which makes TensorFlow.js useful not only for low-latency inference but for privacy-preserving applications as well.
- Even from a mobile device, you can open your Web page, in which case your model can take advantage of sensor data.

Core concepts of TensorFlow.js

TensorFlow.js provides low-level building blocks for machine learning as well as a high-level, Keras-inspired API for constructing neural networks. Let's take a look at some of the core components of the library.

Tensors: The central unit of data in TensorFlow.js is the tensor—a set of numerical values represented as an array of one or more dimensions. A tensor also contains the shape attribute, which defines the array's shape and essentially gives the count of elements per dimension.

The most common way to create a tensor is by using the *tf.tensor* function, as shown in the code snippet below:

```
// 2x3 Tensor
const shape = [2, 3]; // 2 rows, 3 columns
const a = tf.tensor([2.0, 3.0, 4.0, 20.0, 30.0, 40.0],
  shape);
a.print(); // print Tensor values
// Output: [[2,3,4 ],
//          [20, 30, 40]]
```

However, for constructing low-rank tensors, Google advises using the *tf.scalar*, *tf.tensor1d*, *tf.tensor2d*, *tf.tensor3d* and *tf.tensor4d* functions.

The following code example creates an identical tensor to the one shown in the previous code snippet using *tf.tensor2d*:

```
const c = tf.tensor2d([[2.0, 3.0, 4.0], [20.0, 30.0,
40.0]]);
c.print();
// Output: [[2,3,4 ],
//          [20, 30, 40]]
```

TensorFlow.js also provides functions for creating tensors with all values set to 0 (*tf.zeros*) or all values set to 1 (*tf.ones*).

```
// 3x5 Tensor with all values set to 0
const zeros = tf.zeros([3, 5]);
// Output: [[0, 0, 0, 0, 0],
//          [0, 0, 0, 0, 0],
//          [0, 0, 0, 0, 0]]
```

Variables: Variables are initialised with a tensor of

values. Unlike tensors, however, variables are mutable. You can assign a new tensor to an existing variable using the *assign* method, as shown below:

```
const initialValues = tf.zeros([6]);
const biases = tf.variable(initialValues); // initialize
biases
biases.print(); // output: [0, 0, 0, 0, 0,0]

const updatedValues = tf.tensor1d([0, 1, 0, 1, 0,1]);
biases.assign(updatedValues); // update values of biases
biases.print(); // output: [0, 1, 0, 1, 0,1]
```

Operations: While tensors allow you to store data, operations (ops) allow you to manipulate that data. TensorFlow.js provides a wide array of operations suitable for general computations and ML-specific operations on tensors. Because tensors are immutable, operations return new tensors after computations are done—for example, a unary operator such as a square.

```
const data = tf.tensor2d([[2.0, 3.0], [4.0, 5.0]]);
const data_squared = data.square();
data_squared.print();
// Output: [[4, 9 ],
//          [16, 25]]
```

So is the case with binary operations such as *add*, *sub* and *mul*, as shown below:

```
const e = tf.tensor2d([[1.0, 2.0], [3.0, 4.0]]);
const f = tf.tensor2d([[5.0, 6.0], [7.0, 8.0]]);

const e_plus_f = e.add(f);
e_plus_f.print();
// Output: [[6 , 8 ],
//          [10, 12]]
```

TensorFlow.js has a chainable API; you can call operations on the result of operations.

```
const sq_sum = e.add(f).square();
sq_sum.print();
// Output: [[36 , 64 ],
//          [100, 144]]
```

All operations are exposed as functions in the main name space; so you could also do the following:

```
const sq_sum = tf.square(tf.add(e, f));
```

Models and layers: Conceptually, a model is a function that takes some input and produces some output. In TensorFlow.js, there are two ways to create models. You